



LLM API

Week 2 – Day 2 Assignment



JUNE 16, 2026
ANITHA PRAJISH

Contents

Text from Image - Base64..... 2

 Request: 2

 Image: 2

 Response: 3

 Output: 3

Text to Audio 4

 Request: 4

 Response: 4

 Output: 4

Multilingual 5

 Request: 5

 Response: 5

 Output: 5

Generate API test cases from Swagger Code 7

 Request: 7

 Response: 8

 Output 8

Text from Image- Base64

Request:

```
{
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "This is a image. Extract and return only the text exactly as shown, preserving
case."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "data:image/png;base64,{{base64_img}}"
          }
        }
      ]
    }
  ]
}
```

Image:



Response:

The screenshot displays a REST client interface with a sidebar on the left containing a list of collections and environments. The main area shows a POST request to `https://api.groq.com/openai/v1/chat/completions`. The response is shown in JSON format, indicating a successful status of 200 OK. The response body contains a JSON object with fields for `id`, `object`, `created`, `model`, and `choices`. The `choices` array contains a single object with `index`, `message` (including `role` and `content`), `logprobs`, and `finish_reason`.

```
{
  "id": "chatcmpl-08c6a6e2-d2e8-47e6-a299-c9529e11aab8",
  "object": "chat.completion",
  "created": 1781606578,
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "\"LIVE LIFE WITH NO EXCUSES, \\nTRAVEL WITH NO REGRET\\n- OSCAR WILDE\""
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ]
}
```

Output:

"LIVE LIFE WITH NO EXCUSES,

TRAVEL WITH NO REGRET"

- OSCAR WILDE

Text to Audio

Request:

```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
  "voice": "diana",
  "response_format": "wav"
}
```

Response:

The screenshot displays a REST client interface with a sidebar on the left showing a collection of API requests. The main panel shows a POST request to `https://api.groq.com/openai/v1/audio/speech` with a JSON body. The response is a 200 OK status with a 1.50s duration and 402.36 KB size. Below the response details is a large black rectangular area representing the audio output, with a play button and a progress bar at the bottom.

```
POST https://api.groq.com/openai/v1/audio/speech
```

```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
  "voice": "diana",
  "response_format": "wav"
}
```

200 OK • 1.50 s • 402.36 KB

0:08 / 0:08

Output:



LLMAPITexttoAudioR
esponse.mp4

Multilingual

Request:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "Merci beaucoup ! Pourriez-vous m'expliquer comment utiliser cette API ?"
    }
  ]
}
```

Response:

The screenshot displays a REST client interface with a sidebar on the left containing a list of collections and environments. The main area shows a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with a system message and a user message in French. The response is a 200 OK status with a JSON body containing an assistant's reply in French.

```
POST https://api.groq.com/openai/v1/chat/completions
Body
[{"role": "system", "content": "You are an API Specialist. Always respond in the same language as the user."}, {"role": "user", "content": "Merci beaucoup ! Pourriez-vous m'expliquer comment utiliser cette API ?"}]

200 OK • 2.29 s • 1.82 KB
{"index": 0, "message": {"role": "assistant", "content": "Bien s\u00fbr ! Je serais ravi de vous aider \u00e0 comprendre comment utiliser cette API.\n\nTout d'abord, il est important de noter que les API (Application Programming Interface) sont des interfaces de programmation qui permettent \u00e0 diff\u00e9rentes applications de communiquer entre elles. Dans le cas de l'API que vous souhaitez utiliser, il est probable que vous deviez suivre les \u00e9tapes suivantes :\n\n1. **Inscription et cl\u00e9 d'acc\u00e8s** : vous devriez peut-\u00eatre vous inscrire pour obtenir une cl\u00e9 d'acc\u00e8s ou un jeton d'authentification qui vous permettra d'utiliser l'API.\n2. **Documentation de l'API** : lisez la documentation de l'API pour comprendre les endpoints disponibles, les m\u00e9thodes HTTP autoris\u00e9es (GET, POST, PUT, DELETE, etc.) et les param\u00e8tres requis pour chaque endpoint.\n3. **Choix de la m\u00e9thode de requ\u00eate** : s\u00e9lectionnez la m\u00e9thode de"}}
```

Output:

Bien s\u00fbr ! Je serais ravi de vous aider \u00e0 comprendre comment utiliser cette API.

Tout d'abord, il est important de noter que les API (Application Programming Interface) sont des interfaces de programmation qui permettent \u00e0 diff\u00e9rentes applications de communiquer entre elles.

Dans le cas de l'API que vous souhaitez utiliser, il est probable que vous deviez suivre les étapes suivantes :

1. ****Inscription et clé d'accès**** : vous devrez peut-être vous inscrire pour obtenir une clé d'accès ou un jeton d'authentification qui vous permettra d'utiliser l'API.
2. ****Documentation de l'API**** : lisez la documentation de l'API pour comprendre les endpoints disponibles, les méthodes HTTP autorisées (GET, POST, PUT, DELETE, etc.) et les paramètres requis pour chaque endpoint.
3. ****Choix de la méthode de requête**** : sélectionnez la méthode de requête appropriée pour votre besoin spécifique (par exemple, récupérer des données, créer une nouvelle ressource, mettre à jour une ressource existante, etc.).
4. ****Construction de la requête**** : construisez votre requête en incluant les paramètres requis, tels que les en-têtes, les données de formulaire, les paramètres de requête, etc.
5. ****Envoi de la requête**** : envoyez votre requête à l'endpoint de l'API en utilisant la méthode de requête choisie.
6. ****Traitement de la réponse**** : traitez la réponse reçue de l'API, qui peut inclure des données, des erreurs, des messages de succès, etc.

Pouvez-vous me donner plus de détails sur l'API que vous souhaitez utiliser et ce que vous essayez d'accomplir ? Je serais heureux de vous fournir des conseils plus spécifiques.

Generate API test cases from Swagger Code

Request:

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n(1) Generate ONLY Java Rest Assured code.\n(2) Do NOT include explanations, markdown, or extra text.\n(3) Use TestNG as the test runner.\n(4) Use meaningful class, method, and variable names.\n(5) Include imports required for Rest Assured, TestNG, and assertions.\n(6) Use reusable base URI setup.\n(7) Cover CRUD operations for Pet resource.\n(8) Add assertions for status code and important response fields.\n(9) Keep the code clean, executable, and production-style.\n(10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"
    }
  ]
}
```


Response:

The screenshot shows a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is in JSON format, containing a system message and a user message. The response is a 200 OK status with a JSON body containing an assistant message with Java code.

```
POST https://api.groq.com/openai/v1/chat/completions

{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "Generate a REST Assured test for a PetStore API endpoint."
    }
  ]
}
```

Response: 200 OK • 2.62 s • 2.68 KB

```
{
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\n\nimport java.util.HashMap;\nimport java.util.Map;\n\npublic class PetStoreApiCrudTest {\n    private static long petId;\n    private static final String BASE_URI = \"https://petstore.swagger.io/v2\";\n\n    @BeforeClass\n    public void setup() {\n        RestAssured.baseURI = BASE_URI;\n    }\n\n    @Test\n    public void createPet() {\n        petId = System.currentTimeMillis();\n        Map<String, Object> pet = new HashMap<>();\n        pet.put(\"id\", petId);\n        pet.put(\"name\", \"Rex\");\n        pet.put(\"status\", \"available\");\n    }\n}"}
```

Output

```
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.Assert;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
```

```
import java.util.HashMap;
import java.util.Map;
```

```
public class PetStoreApiCrudTest {
```

```
    private static long petId;
    private static final String BASE_URI = "https://petstore.swagger.io/v2";
```

```
    @BeforeClass
    public void setup() {
        RestAssured.baseURI = BASE_URI;
    }
}
```

```
    @Test
    public void createPet() {
```

```

petId = System.currentTimeMillis();
Map<String, Object> pet = new HashMap<>();
pet.put("id", petId);
pet.put("name", "Rex");
pet.put("status", "available");

Response response = RestAssured.given()
    .contentType(ContentType.JSON)
    .body(pet)
    .when()
    .post("/pet")
    .then()
    .extract()
    .response();

Assert.assertEquals(response.getStatusCode(), 200, "Create pet status code");
Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Created pet id");
Assert.assertEquals(response.jsonPath().getString("name"), "Rex", "Created pet name");
Assert.assertEquals(response.jsonPath().getString("status"), "available", "Created pet status");
}

@Test(dependsOnMethods = "createPet")
public void getPetById() {
    Response response = RestAssured.given()
        .accept(ContentType.JSON)
        .when()
        .get("/pet/{petId}", petId)
        .then()
        .extract()
        .response();

    Assert.assertEquals(response.getStatusCode(), 200, "Get pet status code");
    Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Retrieved pet id");
    Assert.assertEquals(response.jsonPath().getString("name"), "Rex", "Retrieved pet name");
    Assert.assertEquals(response.jsonPath().getString("status"), "available", "Retrieved pet status");
}

@Test(dependsOnMethods = "getPetById")
public void updatePet() {
    Map<String, Object> updatedPet = new HashMap<>();
    updatedPet.put("id", petId);
    updatedPet.put("name", "Max");
    updatedPet.put("status", "pending");

    Response response = RestAssured.given()
        .contentType(ContentType.JSON)
        .body(updatedPet)
        .when()
        .put("/pet")
        .then()
        .extract()

```

```

        .response();

        Assert.assertEquals(response.getStatusCode(), 200, "Update pet status code");
        Assert.assertEquals(response.jsonPath().getLong("id"), petId, "Updated pet id");
        Assert.assertEquals(response.jsonPath().getString("name"), "Max", "Updated pet name");
        Assert.assertEquals(response.jsonPath().getString("status"), "pending", "Updated pet status");
    }

    @Test(dependsOnMethods = "updatePet")
    public void deletePet() {
        Response response = RestAssured.given()
            .when()
            .delete("/pet/{petId}", petId)
            .then()
            .extract()
            .response();

        Assert.assertEquals(response.getStatusCode(), 200, "Delete pet status code");
        Assert.assertEquals(response.jsonPath().getString("message"), String.valueOf(petId), "Delete confirmation message");
    }
},

```

"reasoning": "We need to output Java code only, no markdown fences, no explanations. Include imports. Use Rest Assured, TestNG. Provide Base URI setup in @BeforeClass. Provide test methods for CRUD operations. Use meaningful names. Include request body creation (likely using HashMap or POJO). Use assertions for status code and important fields. Should be executable.

We'll need to import `io.restassured.RestAssured`, `io.restassured.response.Response`, `io.restassured.http.ContentType`, `org.testng.annotations.*`, `org.testng.Assert`.

We'll set `baseURI` to `"https://petstore.swagger.io/v2"`. Use static `basePath`? Use endpoint paths.

We'll need a `petId` variable that persists across tests. Use static `long petId`. In `createPet`, generate random id (maybe using `System.currentTimeMillis()`).

We'll create JSON body as a Map with fields: `id`, `name`, `status`. Or create a POJO but Map simpler.

Implement `createPet`: `POST /pet`.

Validate status code 200, response body id matches, name matches.

`GetPet`: `GET /pet/{petId}`. Validate status 200 and fields.

`UpdatePet`: `PUT /pet`. Change name or status. Validate status 200.

`DeletePet`: `DELETE /pet/{petId}`. Validate status 200.

Add `@Test` with `dependsOnMethods` to maintain order.

Add `@BeforeClass` to set `baseURI`.

Use `RestAssured.given().contentType(ContentType.JSON).body(pet).when().post("/pet")` etc.

Ok produce final code.